



SCHOOL OF COMPUTATION, INFORMATION
AND TECHNOLOGY

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

**A Future-proof and
Microarchitecture-agnostic Approach to
Collecting CPU Hardware Samples**

Marius Albrecht



SCHOOL OF COMPUTATION, INFORMATION
AND TECHNOLOGY

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

**A Future-proof and
Microarchitecture-agnostic Approach to
Collecting CPU Hardware Samples**

**Ein zukunftsicheres und
Mikroarchitektur-unabhängiges Verfahren
zum Sammeln von CPU Hardware Samples**

Author:	Marius Albrecht
Supervisor:	Prof. Dr. Martin Schulz
Advisor:	M.Sc. Stepan Vanecek
Submission Date:	11.05.2026

I confirm that this bachelor's thesis in informatics is my own work and I have documented all sources and material used.

Munich, 11.05.2026

Marius Albrecht

Acknowledgments

I sincerely thank my advisor, Stepan Vanecek, for his guidance and support throughout this project. I also thank the Leibniz Supercomputing Centre for providing great compute resources for me to use.

Abstract

Identifying which memory accesses limit a high-performance application’s throughput requires precise, hardware-assisted sampling — the role Intel PEBS and AMD IBS were designed to play. Mitos, originally developed at Lawrence Livermore National Laboratory as the data-collection front-end for the MemAxes visualiser, exposes these facilities to HPC developers: it samples loads above a configurable latency threshold, attributes each sample to a source file and line, and produces an output bundle that downstream tools consume. Its implementation, however, encodes PEBS events as literal opcodes against a specific Intel microarchitecture, which both ties the tool to a small set of supported CPUs and makes supporting new microarchitectures or sampling technologies a manual, error-prone effort.

This thesis re-implements Mitos on top of the Caliper performance instrumentation framework and the libpfm4 PMU-event library, replacing direct `perf_event_attr` construction in the tool itself with symbolic event encoding and the bespoke sampling machinery with a Caliper service, therefore creating a small abstraction layer above architecture-specific encodings. The same source compiles and runs unmodified on Intel Skylake and Intel Ice Lake; the elimination of the Dyninst dependency in favour of Caliper’s symbol-lookup service and an output path compatible with the existing MemAxes pipeline follow as by-products of the redesign. An AMD platform was implemented, but access issues on the available systems prevented the test from completing in time; the IBS half of the cross-vendor claim therefore rests on the design argument alone.

Overall the new implementation is severely lacking compared to the original Mitos. It is briefly argued that the design is sound and the implementation is a first step towards a more robust, future-proof Mitos, but the current state of the codebase is not fit for production use. This argument is not detailed and unverified.

Contents

Acknowledgments	iii
Abstract	iv
1 Motivation	1
2 Background	2
2.1 Processor Microarchitecture	2
2.1.1 Instruction-level parallelism	2
2.1.2 Speculative execution	3
2.1.3 Micro-operations and retirement	3
2.1.4 Memory hierarchy	4
2.2 Hardware Performance Monitoring	4
2.3 Precise Sampling Mechanisms	6
2.3.1 Intel Processor Event Based Sampling	6
2.3.2 AMD Instruction-Based Sampling	7
2.4 libpfm4	8
2.5 Caliper	9
2.5.1 Config and Option Specs	9
2.5.2 Attributes and Snapshots	10
2.5.3 Event Callbacks and Service Lifecycle	10
2.5.4 Service Registration and Static Composition	11
2.5.5 Async-Signal-Safety	11
2.6 Mitos	12
2.6.1 MemAxes	13
2.7 Related work	14
3 Implementation	15
3.1 Design requirements	15
3.2 Technology selection	16
3.3 System overview	18
3.4 Configuration	19

3.5	Sample callback	21
3.5.1	Architectural placement	21
3.5.2	Mechanism	21
3.5.3	Temporal proximity	22
3.5.4	Contract on user callbacks	23
3.5.5	Programmatic interface	23
3.6	Parallel runtime support	24
3.7	Output format and MemAxes integration	25
3.8	Build and deployment	25
4	Evaluation	26
4.1	Methodology	26
4.2	Functional checks (R5–R9)	27
4.3	Data (R1, R2, R7)	28
4.4	Performance (R16, R17)	28
4.5	Microarchitecture independence (R11–R15)	29
4.6	Threats to validity	30
5	Conclusion	31
	List of Figures	32
	List of Tables	33
	Bibliography	34

1 Motivation

High-performance computing (HPC) is a field essential for various scientific, industrial, and governmental applications [1–3] with the main goal of achieving good performance when running these applications.

Available hardware constantly improves, and so software needs to constantly improve and adapt as well. To achieve great performance, developers need to write code that utilizes the hardware efficiently. In many cases overall performance is bound by one specific resource, such as compute power or memory bandwidth. Meaningful optimizations aim to alleviate these bottlenecks so that all resources are used more effectively, increasing overall performance. [4]

Memory-bound workloads are hard to optimize from source code alone: code-centric profilers show where time is spent but not which data accesses caused the stalls or at what level of the memory hierarchy. Closing this gap requires recording per-access addresses, latencies, and memory-hierarchy levels, which modern CPUs expose through facilities such as Intel PEBS and AMD IBS [5, 6].

Mitos [7] turns these raw samples into a workflow developers can act on. Originally built at LLNL as the data-collection front-end for MemAxes [8], it samples loads above a configurable latency threshold, attributes each sample to a source file and line, and bundles the result with a node topology dump. This thesis was based on the assumption that the original Mitos implementation does not properly work on the Intel Ice Lake architecture. Later it turned out that is not true.

This work originally aimed to address this "limitation" by using a more microarchitecture-independent approach to access sampling data. The goal of this thesis changed to providing a more robust, future-proof Mitos implementation.

2 Background

Modern CPUs are designed around aggressive microarchitectural optimizations that decouple the instruction stream from the actual execution behavior in hardware. This complicates performance analysis because the temporal relationship between instruction issue, execution, and retirement is no longer strictly sequential. As a consequence, attributing performance-relevant events to individual instructions requires hardware support beyond simple counting mechanisms. This section provides an overview of relevant CPU microarchitectural features, hardware performance monitoring capabilities, and precise sampling mechanisms.

2.1 Processor Microarchitecture

Modern processors achieve high performance through a number of microarchitectural optimizations that increase instruction throughput and hide memory latency. However, these optimizations also decouple the architectural instruction stream from the actual execution behavior in hardware, which complicates performance analysis and event attribution. Understanding these mechanisms is therefore important when working with hardware-assisted profiling and precise sampling facilities such as Intel PEBS and AMD IBS.

2.1.1 Instruction-level parallelism

Modern CPUs exploit instruction-level parallelism (ILP) to execute multiple instructions concurrently. One important technique for achieving this is pipelining, where instruction execution is divided into multiple stages that can overlap in time [9, C.1]. A simple pipeline can be divided into the following five stages:

1. Instruction Fetch (IF): Fetch the current instruction from memory and advance the program counter
2. Instruction Decode (ID): Decode the instruction and read required operands
3. Execute (EX): Perform arithmetic or logical operations and calculate memory addresses

4. Memory Access (MEM): Access memory for load and store instructions
5. Write Back (WB): Write results back to the register file

Because different instructions can occupy different pipeline stages simultaneously, pipelining increases instruction throughput significantly [9, C.1]. Modern desktop and server processors use much deeper pipelines than the simple example above in order to support higher clock frequencies [10–12].

Modern CPUs further increase parallelism using superscalar execution and out-of-order execution. Superscalar processors can issue multiple instructions per cycle to different execution units. Out-of-order execution allows instructions to execute as soon as their operands become available, rather than strictly following program order [9, 3.4]. Specialized hardware dynamically tracks dependencies between instructions and schedules independent instructions for execution while stalled instructions wait for their operands.

These optimizations improve utilization of execution resources and reduce pipeline stalls. However, they also make it difficult to directly associate hardware events with the original program order, because instructions may execute at different times and in a different order than they appear in the instruction stream.

2.1.2 Speculative execution

Control flow instructions such as branches create uncertainty about which instruction should be fetched next. Waiting until a branch condition has been fully evaluated would introduce pipeline stalls and reduce performance. To mitigate this problem, modern processors use branch prediction algorithms to predict the outcome of branches and continue execution along the predicted path. [9, 3.6]

This prediction enables speculative execution, where instructions are fetched and executed before it is known whether they are actually needed. If the prediction is correct, execution can continue without interruption. If the prediction is incorrect, the processor must discard the speculative work and restore the correct architectural state, which incurs a performance penalty [9, 3.9].

Modern branch prediction mechanisms are highly accurate in practice [13, 14]. Nevertheless, speculative execution further complicates performance analysis because instructions may execute transiently even if they never retire architecturally. Hardware profiling mechanisms must therefore carefully distinguish between speculative and architecturally completed execution.

2.1.3 Micro-operations and retirement

Complex instruction set architectures such as x86 typically do not execute instructions directly in hardware. Instead, instructions are decoded into simpler internal operations

called micro-operations (micro-ops or μ ops), which are then scheduled and executed by the processor core. [9, 3.13] [15, 2.3]

A single architectural instruction can generate multiple micro-operations, especially for memory accesses or complex instructions. These micro-operations may execute independently and out of order inside the processor. Once all operations belonging to an instruction have completed successfully and any exceptions have been resolved, the instruction retires and its results become architecturally visible.

Retirement occurs in program order even if execution itself happened out of order. This distinction is important for performance monitoring because many hardware events are associated either with execution or with retirement. Precise sampling mechanisms such as Intel PEBS and AMD IBS rely on hardware support that allows sampled events to be attributed to the correct architectural instruction despite out-of-order and speculative execution. [5, 2.2.2.3]

2.1.4 Memory hierarchy

Accessing main memory is significantly slower than executing arithmetic or logical instructions. To reduce the resulting performance bottleneck, modern CPUs employ hierarchical memory systems consisting of multiple cache levels [16]. Caches are small but fast memory structures that store frequently accessed data and nearby memory locations [9].

When the processor accesses memory, it first checks whether the requested data is already present in cache (cache hit). If the data is not available (cache miss), it must be fetched from lower levels of the memory hierarchy with increasingly higher latency [9]. Because programs often reuse data (temporal locality) or access memory locations which are close to each other (spatial locality) [17, 18], caches can satisfy a large fraction of memory accesses despite their relatively small size.

Processors additionally employ hardware prefetching mechanisms that attempt to predict future memory accesses and load data into caches before it is explicitly requested [9, 2.2]. While these (and many other) techniques substantially improve average memory performance, memory accesses can still become a major performance bottleneck, especially for applications with irregular access patterns or large working sets [19, 20].

Therefore, understanding memory behavior by profiling memory-related events is crucial for performance analysis and optimization.

2.2 Hardware Performance Monitoring

To monitor performance, processors include performance monitoring units (PMUs) that implement configurable Performance counters (PMCs). PMCs count the number of performance events in hardware, without slowing down what's running on the CPU.

Modern processors support a wide range of events, architectural events which ideally work the same across architectures and non-architectural events which are implementation-specific. PMCs can be set up to trigger an interrupt when the counter overflows, i.e. a threshold number of events have occurred; this is the functionality that is usually used for profiling. [5, 6, 21]

PMUs expose this functionality through a set of registers. Control registers are used to enable or disable individual counters globally. Event select registers determine which hardware or software event a given counter monitors. Status registers report counter capabilities and overflow state. To profile a specific event, the relevant counter is enabled via the control register, its event select register is programmed with the target event, and the counter is preloaded with a negative offset equal to the desired sampling interval so that it overflows after that many events have elapsed. [5, 6, 21]

The Linux `perf_event` subsystem is the standard high-level interface for accessing these hardware features, the `perf_event_open` system call exposes counters as files¹. Counters exposed as such are called either counting or sampling counters. Counting counters simply count the number of occurred events and can be `read` at any time, while sampling counters generate an interrupt every number of events allowing the interrupt handler to collect data: the sample. What's recorded as part of the sample depends on the configuration of the counter, but typically includes the instruction pointer and some additional context information. [22]

The general workflow for sampling using this interface is as follows:

- A) configure PMCs with the threshold and events to monitor
- B) run the application to be profiled
- C) when the threshold is reached and an interrupt generated, the interrupt handler collects data for the sample and records that the threshold was reached
- D) reconfigure the PMCs.

This approach has two key limitations. Because the profiler runs on the same core as the application, they interfere with each other. Increased accuracy through a lower threshold results in more interrupts being raised and more time spent in the handler and less for the application. The profiler running also affects the core's state, for example through memory accesses which pressure the caches.

The second limitation of conventional sampling is the delay between the instruction that triggered the event (and thus an interrupt) and the instruction pointer actually recorded by the interrupt handler and reported in the sample, called *skid*. This effect can

¹Confusingly, the command-line profiling tool "perf" is also part of linux. It's based on this interface.

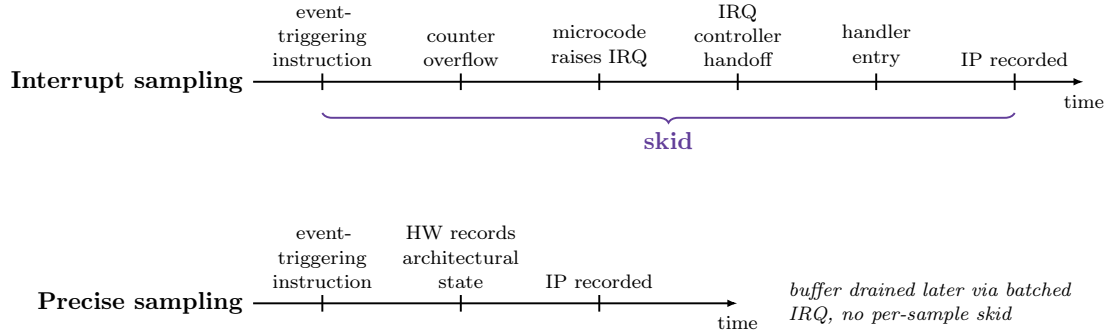


Figure 2.1: Skid in interrupt-based versus precise sampling. In interrupt-based sampling (top), the instruction pointer recorded by the handler is several stages downstream of the event-triggering instruction; the gap is the skid. In precise sampling (bottom), hardware records architectural state at the trigger itself, eliminating per-sample skid.

significantly reduce attribution accuracy and is mainly caused by the latency in handling the interrupt [23]. The interrupt is caused by microcode in the CPU, then has to be handled by the CPU's interrupt controller and only then can software record the sample. All of this takes time, in which program execution continues. [23, 24]

2.3 Precise Sampling Mechanisms

So-called precise sampling addresses this limitation by recording sample data not in software but in hardware, associating recorded samples more directly with the originating instruction [23–25]. While not first-level sources, the respective sections in [26] helped a lot.

2.3.1 Intel Processor Event Based Sampling

Intel Processor Event Based Sampling (PEBS) is a hardware mechanism introduced with the Nehalem architecture that enables precise attribution of sampled events [27]. When a configured counter overflows, PEBS is armed to trap the next time the event occurs. When it does, a microcode routine called PEBS assist records the architectural state directly into a PEBS record in a dedicated memory buffer. Unlike interrupt-based sampling, no skid is introduced. [5, 23]

These records accumulate in a buffer (the DS area) until a programmable buffer threshold is reached. At that point a hardware interrupt occurs, the kernel reads out all

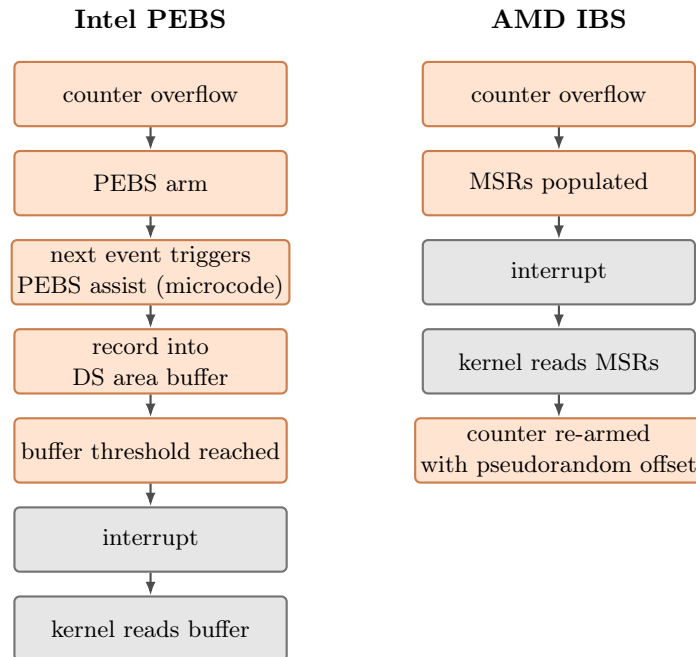


Figure 2.2: PEBS (left) and IBS (right) sample data flow. Shared colour coding distinguishes hardware-recorded (orange) from software- or kernel-handled stages (grey). Both backends converge at the same kernel entry point and expose comparable sample semantics to libpfm4, which is the contract that lets calimitos treat them as interchangeable backends.

PEBS records, and delivers them to the profiling process. [5]

[parsep=5pt]

The exact data recorded in a PEBS record depends on the event being monitored and the processor generation, but includes at least the instruction pointer, timestamp, and thread ID². Some events also record additional information such as the memory addresses accessed, where in the memory hierarchy the accessed data actually was and the latency of said accesses. [5].

2.3.2 AMD Instruction-Based Sampling

AMD’s Instruction Based Sampling (IBS) is a dedicated hardware facility, separate from the general-purpose PMU infrastructure, introduced with the Family 10h generation

²A stream ID, the CPU (commonly also called core outside of the linux context), the current sampling period and the transaction type are also recorded but less important in this work.

of cores [28, 29]. It follows the instruction sampling approach proposed in [30]. Each core’s IBS unit consists of two control registers, two internal counters, and a set of Model Specific Registers (MSRs) that hold sampled data [6].

IBS is divided into two components, which are programmed through the two control registers. IBS Fetch samples instruction fetch behavior, including instruction cache and TLB activity. This enables analysis of front-end bottlenecks such as instruction delivery stalls. IBS Op samples micro-operations, including data cache behavior, memory latency, and execution timing. [6]

After programming the control register, the monitored event will be counted by the respective internal counters belonging to the programmed sampling type (fetch or op counter) in the processor core. Samples are recorded in the MSRs when the counter overflows. A hardware interrupt is then raised, the kernel interrupt handler copies the MSR contents to a kernel-space buffer, and IBS is re-enabled with a fresh pseudorandom counter offset before resuming. [6]

Through the `perf_event_open` interface, the same information can be obtained from IBS samples as from PEBS samples, including the instruction pointer, timestamp, thread ID, and additional context such as memory addresses and latency for certain events³ [6].

2.4 libpfm4

libpfm4 is an open-source library, that translates human-readable PMU event specifications into the binary encodings required by the OS interface of either Linux, macOS or Windows. On Linux its central role is the translation of human-readable PMU event specifications into the binary encodings required by the `perf_event_open()` system call. The library supports a very wide range of PMUs, including most modern AMD, Intel, ARM and IBM architectures, as well as some SPARC and MIPS ones. [31, 32]

libpfm4 is used in code by passing an event string to `pfm_get_os_event_encoding()`, which populates a `perf_event_attr` struct and returns any relevant metadata. The resulting structure can be passed directly to `perf_event_open()` without further modification. [31]

To do so, libpfm4 maintains a compiled-in descriptor table for each supported PMU. Each table enumerates the available events alongside their raw opcodes, unit masks, and modifiers. Unit masks select a subtype of a base event, for example a base memory event can be combined with a mask that gates sampling on access latency `MEM_TRANS_RETIRED:LATENCY_ABOVE_THRESHOLD`. Modifiers control additional sampling attributes such as privilege level filtering or, on Intel, whether PEBS is enabled for a given event. In particular, this means differences in the encoding of the same event between

³For more detail see [6] or for better readability see [29].

microarchitectures are handled by `libpfm4`. [31]

`libpfm4`'s scope is deliberately narrow: it handles event encoding only and has no involvement in file descriptor management, ring buffer configuration, multiplexing, or sample parsing. This makes it a natural low-level dependency for higher-level tools, including PAPI [33] and Caliper [34], that own the `perf_events` lifecycle but want to delegate the non-trivial problem of portable PMU encoding.

2.5 Caliper

Caliper is a performance instrumentation and measurement library developed at Lawrence Livermore National Laboratory (LLNL) [34,35], designed as a composable framework for collecting hardware and software performance data in HPC applications. It supersedes LLNL's Mitos [7,8], which was a narrowly scoped tool for hardware memory access sampling via PEBS. In Caliper this generalizes into a broader instrumentation framework.

Caliper's architecture centers on modular plugins, called services, that implement specific measurement concerns and compose without explicit coupling. Services are activated per measurement channel and communicate through a shared data model of named, typed attributes and snapshots. The `libpfm` service uses `libpfm4` to configure and open `perf_events` sampling descriptors, injecting hardware sample records as snapshots. The `symbollookup` service annotates those samples with function names, source files, and line numbers by consulting symbol tables and DWARF debug information. The `recorder` service persists snapshots to Caliper's native `cali` format, which encodes measurement data alongside metadata required to interpret it; the `caliperreader` Python API allows parsing the `cali` format for flexible downstream analysis. [35,36]

Caliper also provides built-in support for MPI [37], OpenMP [38], Kokkos [39], CUDA [40], ROCm [41], and integrates with third-party tools like NVidia Nsight/NVProf [42,43] and Intel VTune [23]⁴ [36, Building Caliper, Connection to third-party tools]

2.5.1 Config and Option Specs

Caliper can be configured programmatically via the `ConfigManager` C++ API, via persistent configuration files, or through `CALI_CONFIG_*` environment variables, allowing measurement setup to be decoupled from the application build [36].

The `ConfigManager` API allows users to register new measurement configurations and options at runtime by passing JSON specification strings to `add_config_spec` and `add_option_spec`, respectively `citeCaliperDocs`. These must be registered before any channel is created.

⁴This happens through the following services: `mpi`, `mpireport`, `ompt`, `kokkostime`, `kokkoslookup`, `cupti`, `cuptitrace`, `roctracer`, `rocprofiler`, `roctx`, `nvtx` and `vtune`.

A Config Specification (`config_spec`) defines a named measurement configuration used by a `ChannelController`. It is a JSON dictionary specifying the `name` and `description` of the configuration, the Caliper `services` it requires, a `config` dictionary mapping `CALI_*` environment-variable keys to values, a `categories` list that controls which option specs are applicable to this config, `defaults` for any of its options, and an inline `options` list for config-local options. When the required services are not present in the Caliper installation, the configuration is silently unavailable.

An Option Specification (`option_spec`) defines a named configuration modifier that can be applied to any config that lists the option's `category` in its `categories` field. Its JSON dictionary specifies the `name`, `category`, `description`, required `services`, and a `config` dictionary of additional `CALI_*` variables the option activates. Options intended for aggregation-based metrics additionally carry a `query` dictionary with `local` and `cross` CalQL expressions that compute process-local and cross-process metrics from snapshot data, respectively.

Once registered, a config is instantiated by name through `ConfigManager::add`, which accepts an option list in parentheses, e.g. `mgr.add("myconfig(myoption=true)")`, and the resulting channel is started with `ConfigManager::start`. This mechanism is the standard extension point for adding measurement configurations without modifying Caliper's source. [35,36]

2.5.2 Attributes and Snapshots

The core data model in Caliper revolves around attributes and snapshots. An attribute is a named, typed key that services can associate with measurement events. Each attribute is modeled by an `Attribute` object, obtained via `Caliper::get_attribute(const char* name)` or created via `Caliper::create_attribute(name, type, properties)`. Attributes are global to the process and must be created before they are used in snapshots.

A Snapshot is an immutable collection of key-value entries, where each entry pairs an `Attribute` with a `Variant` (a tagged value that can hold integers, floats, strings, etc.). Snapshots are constructed incrementally via a `SnapshotBuilder` object. Entries are added via the `SnapshotBuilder::append(Attribute, Variant)` method. After that, the `SnapshotBuilder` gets implicitly converted to a read-only `SnapshotView`, which allows services to query entries via `SnapshotView::get(Attribute)` or iterate over all entries.

2.5.3 Event Callbacks and Service Lifecycle

Caliper's event system allows services to register callbacks on a variety of measurement events. Key events include:

- **post_init_evt**: Fired after a channel is fully initialized and all services have been registered. Used for service setup that depends on other services' attributes.
- **snapshot**: Fired synchronously when a snapshot is being constructed, before it is finalized. Called with the `SnapshotBuilder` in a mutable state, allowing services to append additional entries.
- **finish_evt**: Fired when a channel is shutting down. Used for resource cleanup.

Services register callbacks by obtaining a reference to the `Channel::events()` object and calling `connect(callback)` on the desired event⁵. Callbacks for the `snapshot` event receive three parameters: a pointer to the active `Caliper` instance, a `SnapshotView` containing entries already added by earlier services, and a `SnapshotBuilder&` through which new entries can be appended. [35]

The snapshot fires synchronously during `Caliper::push_snapshot()`, which may be called from interrupt handlers (e.g., the libpfm SIGIO handler) without special precautions. All snapshot callbacks thus execute in the same signal context as the triggering event, ensuring tight temporal coupling between the sample and any measurements taken in the callback. [35]

2.5.4 Service Registration and Static Composition

Services are composable plugins loaded at runtime via a static service registry, compiled into the Caliper library at build time. Each service exports a descriptor, a `CaliperService` struct containing a JSON specification and a `ServiceRegisterFn` callback. The JSON spec declares the service's name, description, and configuration options. The register function is invoked once per channel that enables the service and is responsible for creating any service-specific `Attribute` objects, connecting callbacks to channel events (Section 2.5.3), and maintaining per-channel instance state if needed. [35]

The build system generates the service registry from declarations added via the CMake macro `add_caliper_service()`, ensuring all services are available without requiring application code to explicitly link or initialize them. This allows services to be enabled purely through configuration, decoupling measurement setup from the application. [35]

2.5.5 Async-Signal-Safety

Because snapshot callbacks execute in signal context (Section 2.5.3), Caliper provides a signal-safe instance accessor, `Caliper::sigsafe_instance()`, that permits querying the Caliper instance from signal handlers without triggering unsafe operations like mutex

⁵For example like this: `channel->events().snapshot.connect(callback);`, given a valid `channel` pointer.

acquisition. All registered callbacks must be async-signal-safe, using only POSIX-approved operations (e.g., `clock_gettime`, atomic reads). This contract is enforced by convention: the programmer is responsible for ensuring their callbacks adhere to signal-safety constraints.

2.6 Mitos

Mitos is a memory-access sampling library and command-line tool, originally developed at LLNL by Giménez et al. as the data-collection front-end for MemAxes [8]. It collects sampled memory performance data on a running application and persists them in a format that MemAxes can consume for visualization. By default, Mitos configures Intel PEBS (Section 2.3.1) to sample loads exceeding a configurable latency threshold and stores; an alternative AMD IBS backend (Section 2.3.2) can be selected at build time [44, 45]. [7]

Mitos exposes three interfaces for invoking the sampler [7]. All three interfaces converge on the same underlying sampler and produce the same on-disk output format.

- **mitosrun** is a wrapper binary that launches a single-threaded target process under sampling and writes its output directory once the target exits.
- **mitoshooks** is a shared library that the target application links against; it transparently starts and stops sampling around OpenMP worker threads or MPI ranks (see below).
- A C API can alternatively be called directly from instrumented application code, for example to bracket a region of interest.

The sampler is a thin wrapper around the Linux `perf_event_open` interface (Section 2.2). For each monitored event Mitos populates a `perf_event_attr` structure manually, opens a file descriptor, maps a kernel ring buffer into user space via `mmap`, and installs a `SIGIO` signal handler that drains completed records from the ring buffer once the kernel signals readiness [22]. The `perf_event_attr` encodings are hard-coded in the source: in PEBS mode, the raw event corresponding to `MEM_TRANS_RETIRED:LATENCY_ABOVE_THRESHOLD` is configured. In IBS mode, the IBS Fetch or IBS Op PMU type is selected by a compile-time flag [7, 45]. Because these encodings are written as literal opcodes against a specific microarchitecture, Mitos’s PEBS path is tied to the Intel generations on which those encodings remain valid.

OpenMP support is implemented through the OpenMP Tools Interface (OMPT) [46]. **mitoshooks** registers `ompt_callback_thread_begin` and `ompt_callback_thread_end` callbacks; the begin callback creates a per-thread output context and starts an independent sampler bound to the OMP worker thread, while the end callback flushes the corresponding

ring buffer. Per-thread raw outputs are merged into a single result directory in a separate post-processing step invoked by the user after the application exits. [7, 45]

MPI support relies on the MPI profiling interface (PMPI) [37]. `mitoshooks` provides hook implementations of `MPI_Init` and `MPI_Finalize` (along with several point-to-point and collective routines for tracing) that forward to the underlying `PMPI_*` symbols. The `MPI_Init` hook starts a Mitos sampler on the local rank, while the `MPI_Finalize` hook ends it, performs source-attribution, and merges per-rank output directories on rank 0 [7, 45].

The output of a Mitos run is a directory `mitos_<timestamp>/` containing the merged samples in CSV form, a `hwloc` [47] dump of the node’s hardware topology (`hardware.xml`), and a copy of the application source files used in the build. Source-line attribution of samples is performed in post-processing using `Dyninst` [48]: the application records the virtual-address offset of its text segment to a temporary file at startup, and Mitos combines this offset with the symbol and DWARF information extracted from the debug-built binary to map each sample’s instruction pointer to a source file and line [7]. The resulting directory layout is the input format expected by `MemAxes`.

Although `MemAxes` (Section 2.6.1) was Mitos’s original target consumer, Mitos has since become a sampling building block for several downstream workflows rather than a standalone tool. The `hwloc` topology dump it emits is read by `sys-sage` [49], a system-topology management library that exposes a queryable representation of static and dynamic node properties to other tools in the HPC stack. A second example is the per-MPI-call CXL.mem performance model of Vanecek et al. [45], which extends Mitos with MPI-aware sampling and PAPI [33] counter collection in order to predict the benefit of replacing message-passing with shared CXL.mem buffers.

2.6.1 MemAxes

Mitos’s primary application is supplying data for `MemAxes`.

"`MemAxes` is a tool for interactive analysis and visualisation of data movements within a node. It uses Mitos output (or possibly output of other tools) as input, and processes the data. `MemAxes` offers an interactive graphical user interface showing how many and how costly data load operations of each CPU core were, at which memory level they were satisfied, and where they originate in code, which enables users to find and mitigate (data movement-related) bottlenecks of their applications. Moreover, there are other views available, showing the sample distribution in time, processes, addresses, or core IDs, to name a few examples." [50]

The available views are the Hardware Topology View, the Parallel Coordinates View and the Code View. The Hardware Topology View visualizes the node’s memory hierarchy and the distribution of samples across it. The Parallel Coordinates View visualizes all

sample attributes in a single plot, allowing users to identify correlations between attributes. For example a big number of samples with a high latency, a specific memory level and a specific core ID could indicate a bottleneck in the memory hierarchy of that core. The Code View shows which source files and source lines are associated with the most memory access cycles. It also offers a code editor which shows relevant source code when the user selects something in any of the views. [8, 51]

Beyond memory samples, this functionality requires:

- a detailed model of the node’s hardware topology, including the memory hierarchy and core layout, in the format provided by hwloc [47]
- samples associated with source files and lines
- source code files for the application which was profiled

2.7 Related work

This work sits at the intersection of three bodies of literature: memory-access profiling tools, precise-sampling research on commodity hardware, and the Caliper measurement-library ecosystem.

Original Mitos [7] was developed by Giménez et al. at LLNL [8] as the data-collection front-end for MemAxes. It was later extended at TUM to support AMD IBS, among other advancements [44, 45]. Section 2.6 and Section 2.6.1 describe both in depth; together they define the workflow this thesis re-implements end-to-end. Schenk [51] extended the MemAxes side of that pipeline to consume AMD IBS samples in addition to PEBS, in parallel to the AMD-IBS extension to Mitos itself.

Intel VTune [23] and AMD uProf [24] are the canonical vendor-supported tools for hardware-assisted profiling on their respective platforms. Both ship rich sample analysis, polished UIs, and proprietary extensions on top of the same precise-sampling facilities (PEBS and IBS, respectively) that new Mitos exposes through libpfm4. Their closed-source, vendor-locked nature is precisely what motivates an open, vendor-agnostic alternative; new Mitos is far narrower in scope, but lives in that complementary niche.

PAPI [33] is the long-standing portable interface for hardware performance counters on HPC systems, used by many higher-level tools as a thin abstraction over `perf_event_open`.

libpfm4 [31] is the descendant of perfmon2 [32] and is the de-facto symbolic encoder for PMU events on Linux, used by PAPI, Caliper, and many other tools. New Mitos’s microarchitecture-independence claim (Section 4.5) ultimately rests on libpfm4’s compiled-in PMU descriptor tables doing the per-microarchitecture encoding work for free; the technology selection of Section 3.2 is, at its core, a decision to delegate to this layer rather than re-implement it. [parsep=5pt]

3 Implementation

Throughout this chapter, the term *original Mitos* refers to the existing tool described in Section 2.6, and *new Mitos* refers to new Mitos presented here.

3.1 Design requirements

The following design requirements will be used to guide the design and implementation of new Mitos and to evaluate it afterwards.

Comparable data The data within the collected samples is the basis for all analysis done with *mitos* and therefore must at least be comparable to the data collected with the original Mitos.

R1 The collected data must not be significantly different in terms of quality compared to the original Mitos.

R2 The collected data must not be significantly different in terms of quantity compared to the original Mitos.

Simple user interface A key aspect of *mitos* is its ease of use and the low barrier to entry for users. This should not change with the new Mitos.

R3 The user interface must not be harder to understand and use than the one in the original Mitos.

R4 The user interface should be close to the one in the original Mitos.

Feature parity New Mitos can only be a viable replacement for original Mitos if the same (or better) features are available. Therefore, feature parity is essential.

R5 Collecting samples from a code parallelized using OpenMP must be easily possible.

R6 Collecting samples from a code parallelized using MPI must be easily possible.

R7 Collected samples must be attributable to source code when the application is compiled with debug symbols.

R8 Specifying a custom sample callback function which can attach arbitrary data to each sample must be possible.

R9 The output format must be compatible with MemAxes [51].

R10 Similarly fine-grained control over sampling and annotating code should be available via an API.

Microarchitecture agnostic and Future proof A design which keeps future hardware developments in mind means implementation effort and necessary code maintenance is reduced and extension to other hardware platforms is easy, should the need arise.

R11 The system should be as independent of the specifics of any Vendor's implementation of precise sampling as possible.

R12 Intel PEBS and AMD IBS must be supported.

R13 In particular, PEBS must be supported on the Icelake microarchitecture.

R14 The system should be designed in a way that is very likely to support future CPU architectures with little effort and without fundamental changes to the design.

R15 Precise sampling technologies similar to PEBS and IBS from other vendors should be easy to integrate.

No performance regression The relatively low performance overhead incurred during profiling is an important aspect of Mitos and one of its key advantages over other profiling tools. Therefore, new Mitos should not cause a significant increase in overhead compared to original Mitos.

R16 Memory usage must not increase significantly compared to the original Mitos.

R17 Execution time must not increase significantly compared to the original Mitos.

3.2 Technology selection

Towards these goals, available technologies to access PMU data are compared and the best fitting one is chosen as a basis for the new Mitos. The main candidates are

- A) The original Mitos with direct usage of the `perf_events` API and, in particular, manual `perf_event_attr` configuration.
- B) The original Mitos but instead of manual `perf_event_attr` configuration, libpfm is used.
- C) Caliper using the libpfm service.

Many of the requirements are either clearly met or not met and don't have a clear path towards meeting them. For this reason I will only discuss the controversial ones in detail. The full comparison can be found in Table 3.1.

Requirements [R1](#) and [R2](#) (data similar quantitatively and qualitatively) are not met for *mitos* with *libpfm* (candidate [B](#)) and Caliper (candidate [C](#)) because this approach is not yet implemented. But, because *libpfm* ultimately uses the same mechanism as candidate [A](#) to access PMU data, both candidates provide a clear path towards meeting these requirements.

Requirement [R8](#) (sample callback) is met for candidates [A](#) and [B](#). Caliper doesn't have sample callback functionality. However, the functionality can be implemented using a custom service and the snapshot event callback mechanism described in Section [2.5.3](#). By doing so, little changes to existing Caliper code are necessary and future updates should be easily mergeable. This means candidate [C](#) has a clear path towards meeting requirement [R8](#).

Requirement [R9](#) (MemAxes compatibility) is met for candidates [A](#) and [B](#), while candidate [C](#) provides different output formats by default. However, the `calireader` Python API can be used to convert the sample data into a MemAxes compatible *csv* format. Alongside the sample data MemAxes also expects data about the hardware topology of the system, as provided by *hwloc*. This can be run alongside Caliper, just like in the original *Mitos*. This means candidate [C](#) has a clear path towards meeting requirement [R9](#).

Requirements [R11](#), [R12](#) and [R13](#) (architecture independence, general support for IBS and PEBS) are not met for candidate [A](#) because the original *Mitos* uses hard-coded manual PMU configuration. Candidate [C](#) meets the requirements because of its *libpfm* service. Similarly, candidate [B](#) has a clear path towards meeting requirements [R11](#), [R12](#) and [R13](#) by implementing configuration using *libpfm4*.

Because the original *Mitos* (candidate [A](#)) doesn't meet requirements [R11](#), [R12](#) and [R13](#), it is not a viable basis for the new *Mitos*. Both *mitos* with *libpfm* (candidate [B](#)) and Caliper (candidate [C](#)) meet many but not all requirements but for both of them there is a clear path towards meeting all requirements.

Therefore, both of them are reasonable bases for the new *Mitos*. I decide to use Caliper because of the following reasons:

- Caliper is a mature project with multiple active developers and the Lawrence Livermore National Laboratory as its developing organization [\[35\]](#), which means that it is likely to be well-maintained and supported in the future.
- Caliper has a wide variety of features outside the scope of this project, which would also become accessible to users of the new *Mitos* and therefore increase its potential use cases (Section [2.5](#)).

	bare (A))	libpfm (B))	Caliper (C))
R1	yes	possible	possible
R2	yes	possible	possible
R3	yes	yes	yes
R4	yes	yes	yes
R5	yes	yes	yes
R6	yes	yes	yes
R7	yes	yes	yes
R8	yes	yes	possible
R9	yes	yes	possible
R10	yes	yes	yes
R11	no	possible	yes
R12	no	possible	yes
R13	no	possible	yes
R14	no	yes	yes
R15	no	yes	yes
R16	yes	yes	yes
R17	yes	yes	yes

Table 3.1: Comparison of the different technology candidates towards the design requirements. "yes" means the requirement is met, "possible" means it is not met but there is a clear path towards meeting it and "no" means it is not met and there is no clear path towards meeting it.

3.3 System overview

New MitoS is delivered as a single header file, `calimitos.h`, together with a fork of Caliper that contributes one new service. No additional shared library is introduced: the application links against the (forked) Caliper installation and includes the header directly. Because new MitoS ideally uses the same API as original MitoS's C API, a header-only adapter is the smallest possible footprint that retains binary-comparable user-facing semantics. Requirements **R3** and **R4** (interface ease of use, interface continuity) benefit directly from this choice. A consequence of the header-only design is that state shared across translation units must be declared `inline` (C++17) to obtain a single shared instance rather than a per-translation-unit copy [52]. All implementation state therefore resides in a `calimitos_detail` namespace as `inline` variables; the public `MitoS_*` functions and `backend_t` type are re-exported at global scope via `using` declarations, keeping the plain `MitoS_*` functions.

The functional decomposition aims to mirror the structure of original MitoS as described

in Section 2.6, with each prior component replaced by a corresponding Caliper service:

- Direct `perf_event_attr` configuration is replaced by the `libpfm` service, which composes `libpfm4` (Section 2.4) with Caliper’s snapshot infrastructure.
- The SIGIO signal handler and ring-buffer drain are likewise provided by the `libpfm` service.
- Source attribution, performed in original Mitos as a Dyninst-based postprocessing step, is performed by the `symbollookup` service.
- The CSV writing is replaced by the `recorder` service together with the `cali_to_csv` Python converter described in Section 3.7.
- The user sample handler is replaced by a custom `sample_callback` service (Section 3.5). Users can register their own callback functions, similar to the original Mitos API.

Almost the entire stack is replaced by composed Caliper services. The subsequent sections describe the small amount of glue and the single new service required to assemble these components into a Mitos-shaped tool.

3.4 Configuration

The measurement configuration is assembled by subclassing Caliper’s `ChannelController`. The `MitosChannelController` subclass packages the entire configuration and handles configuration semantics that can’t be expressed through the standard Caliper API (Section 2.5.1). Under the hood, a configuration map with `CALI_*` keys is used.

There are two principal dimensions of configuration that the controller handles: First which sampling backend (and thus which PMU event) to use, and second whether an MPI-aware configuration is desired. Backend selection is automatically done at runtime by inspecting the `flags` line of `/proc/cpuinfo` for the substrings `pebs` and `ibs` but the user can, and in many cases should, set all relevant configuration values themselves through an explicit API call.

The same events which used in original Mitos are used for the PEBS, IBS Op, and IBS Fetch backends. For PEBS that is the `libpfm MEM_TRANS_RETIRED:LOAD_LATENCY` event string. Original Mitos uses `MEM_TRANS_RETIRED:LATENCY_ABOVE_THRESHOLD`, which is an alias on some architectures but doesn’t exist on others [31]. The threshold and the desired attributes per sample are configured in the `ChannelController`, because backend selection happens at runtime and can therefore not be handled through an `Option`. The threshold value is specific to this event. The IBS Op backend selects `amd64_ibs_op::IBS_OP_ALL`;

the IBS Fetch backend selects `amd64_ibs_fetch::IBS_FETCH_ALL` with the `IbsRandEn` bit (bit 57) set in `CALI_LIBPFM_CONFIG` to randomize the fetch counter, matching the original Mitos `USE_IBS_FETCH` default [6, 13.3.2]. All events are specified symbolically rather than as raw opcodes. The resulting encoding portability across microarchitectures within a given vendor is provided entirely by `libpfm4` (Section 2.4). This portability is the mechanism by which R11, R13, and R14 are met: the implementation contains no microarchitecture-specific opcodes, and adapting to a new microarchitecture of the same vendor requires no source change¹.

All relevant configs can also be manually set by the user through the API described in Table 3.2. The API is a wrapper around the mechanism described above.

`Mitos_enable_mpi` needs to always be manually called. The sampling itself does work without it but the output will not be in the required format. `Mitos_set_backend` does not necessarily need to be called but the user should explicitly make sure the correct configuration is selected. The automatic selection works in many cases but is not robust. `Mitos_set_handler_fm` needs to be called when required; see Section 3.5 for details. The frequency, period and latency threshold smodify the sampled event. Deviation from the defaults may be necessary depending on the usecase. If both frequency and period are set, the one set last takes precedence. The recorder configs and the `cali_to_csv` config should not usually be necessary.

Call	Description
<code>Mitos_enable_mpi</code>	Pull in MPI services; tag samples with <code>mpi.rank</code>
<code>Mitos_set_backend</code>	Select backend: <code>backend_auto</code> , <code>backend_pebs</code> , <code>backend_ibs_op</code> , <code>backend_ibs_fetch</code>
<code>Mitos_set_sample_event_period</code>	Sample every p events (mutually exclusive with frequency)
<code>Mitos_set_sample_time_frequency</code>	Sample at frequency f (mutually exclusive with period)
<code>Mitos_set_sample_latency_threshold</code>	PEBS-only: minimum latency in cycles to record
<code>Mitos_set_handler_fn</code>	Register a per-sample callback
<code>Mitos_set_recorder_directory</code>	Override the output directory for <code>.cali</code> traces
<code>Mitos_set_recorder_filename</code>	Override the output filename for <code>.cali</code> traces
<code>Mitos_set_cali_to_csv_script</code>	Path to <code>cali_to_csv.py</code> ; overrides <code>CALIMITOS_CALI_TO_CSV</code>

Table 3.2: new Mitos configuration API. The calls are ranked in terms of importance.

¹It does require source changes but they'll be in `libpfm4`. An update of `libpfm4` or Caliper might be necessary.

3.5 Sample callback

To satisfy requirement R8 (sample callback) a new Caliper service, `sample_callback`, is implemented. The service provides an API for registering a callback function that is, in turn, used by `Mitos_set_handler_fn`. An implementation sketch was discussed in Section 3.2, when choosing Caliper as a backend. The details of the implementation and the design decisions are described subsequently.

3.5.1 Architectural placement

The implementation is carried as a fork of Caliper. Any modification to existing upstream files would become a recurring merge conflict on every future release. Two candidate placements were considered for the callback dispatch:

1. direct modification fo the libpfm service's `sample_handler` to invoke the user callback;
2. a self-contained new service in its own subdirectory under `src/services/`.

I selected the new service as the minimum-impact mechanism consistent with the upstream-tracking constraint. The only modification to existing files is a single line in `src/services/CMakeLists.txt` to register the new service's source files with the build system. The new service is otherwise entirely self-contained, with no upstream merge conflicts expected.

3.5.2 Mechanism

The service occupies a single position in the channel's `snapshot` callback chain. Its `ServiceRegisterFn` registers with `chn->events().snapshot`. The callback is invoked synchronously by `Caliper::push_snapshot` for every snapshot taken on the channel, with the signature `void(Caliper*, SnapshotView trigger, SnapshotBuilder& rec)`. This exact signature is fully forwarded to the user. The `trigger` `SnapshotView` contains the entries already attached by the triggering service (in the case of a libpfm sample, the entries assembled in the libpfm `sample_handler`) and the `rec` builder is mutable, accepting further entries via `rec.append(attribute, value)`.

The service exposes a `void register_callback(function, args)` function along with the `callback_fn` function type, both in the `cali::sample_callback` namespace. This registration function is used by `Mitos_set_handler_fn` to register the user callback with the service. To do this, the `sample_callback.h` header is included.

The `snapshot` event also fires for triggers other than libpfm samples, for example region begin and end events propagated by the `event` service, or explicit `Caliper::push_snapshot`

calls from other services. By default the dispatcher invokes the user callback on every snapshot. To suppress invocations on snapshots for which the user callback has no meaningful work, the service exposes a `CALI_SAMPLE_CALLBACK_FILTER_ATTRIBUTE` configuration variable: when set, the dispatcher inspects the `trigger` view and invokes the user callback only when an entry with the named attribute is present. New Mitos sets this variable to `libpfm.event_sample_name` in `MitosChannelController`, so within a Mitos channel the user callback is invoked exactly on libpfm samples. This indirection is what keeps the new service free of any hard-coded dependency on the libpfm attribute namespace; the libpfm coupling lives entirely in `calimitos.h`, not in the Caliper service itself, which preserves the future-proofing argument of Section 3.1 (R11, R14). Filtering is applied in the hot path with two atomic loads, one branch on the cached `Attribute` object, and a small linear scan over the trigger view; when filtering is disabled, the second branch short-circuits.

3.5.3 Temporal proximity

The principal property delivered by this placement is that the user callback executes within the same signal handler invocation as the kernel sample delivery. The call stack at the point of invocation, from the bottom up, is: the kernel SIGIO delivery; the libpfm service’s `sigio_handler`; the libpfm service’s `sample_handler`; `Caliper::push_snapshot`; and the dispatcher in the `sample_callback` service. The temporal coupling between the

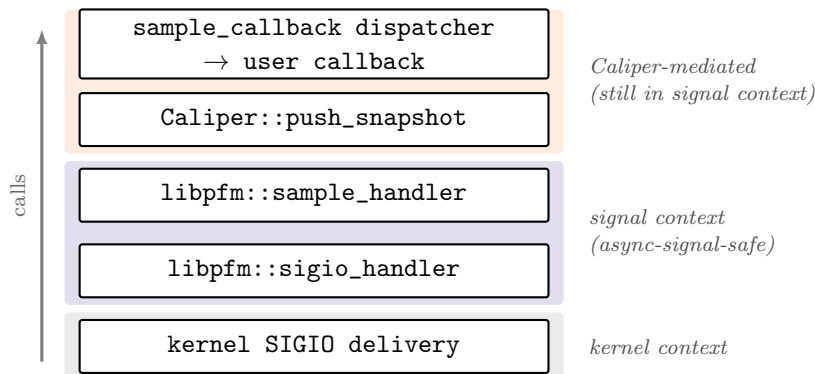


Figure 3.1: Call stack at the moment the user callback is dispatched. Bottom-up: the kernel delivers SIGIO; the libpfm service’s `sigio_handler` and `sample_handler` run in signal context; `Caliper::push_snapshot` drives the snapshot pipeline; and the `sample_callback` dispatcher invokes the user callback. All frames above the kernel run in the same signal-handler invocation, so the user callback inherits the async-signal-safety contract of Section 2.5.5.

hardware event and the user callback is bounded only by the latency that the libpfm service itself incurs between sample arrival and the call to `Caliper::push_snapshot`. Similarly, the original MitoS also processes the entire sample before the custom callback is invoked.

3.5.4 Contract on user callbacks

Because the user callback executes in signal context, it is bound by the async-signal-safety contract introduced in Section 2.5.5. The contract is enforced by convention rather than by mechanism: the set of permissible operations corresponds to the POSIX async-signal-safe function list [53], and the `Caliper::sigsafe_instance()` accessor must be used in place of `Caliper::instance()` whenever the callback queries Caliper state.

A second, related constraint follows. Any `Attribute` object used in `rec.append(attr, value)` must be created outside the callback via `Caliper::create_attribute`, because that call is itself not async-signal-safe. `Attribute` objects are cheap to copy and may be retained as global or thread-local variables for use inside the callback.

These constraints fix a calling order. `Caliper::create_attribute` for every attribute the callback will append, and `MitoS_set_handler_fn` itself, must precede `MitoS_begin_sampler`; once sampling is active, the only Caliper operations the callback may perform are signal-safe lookups against pre-created attributes and `rec.append` into the supplied `SnapshotBuilder`. Registering the callback after sampling has started leaves the initial samples flowing through the default no-op dispatcher.

3.5.5 Programmatic interface

The registration interface consists of a single function and a single typedef:

- a callback type `cali::sample_callback::callback_fn`, defined as a function pointer with signature `void(Caliper&, SnapshotView, SnapshotBuilder&, void*)`;
- a registration entry point `MitoS_set_handler_fn(fn, args)` that installs the callback together with an opaque `void*` user context which will get handed to the callback.

The callback pointer and its argument are stored as `std::atomic` variables internal to the service, so that the dispatcher can load them safely from signal context. The choice of a plain function pointer in preference to `std::function` is itself a signal-safety consequence: a `std::function` invocation is not generally async-signal-safe, whereas an atomic load of a function pointer followed by an indirect call is signal-safe by construction.

A typical use of the interface follows a fixed pattern. The application creates the attributes it intends to attach, registers its callback, and then begins sampling. Inside

the callback, it uses only async-signal-safe operations to compute the data to attach and appends it to the supplied builder. The interface places no restriction on the semantic content of the attached data; the attribute names, types, and values are entirely under the control of the application. Though, because the callback executes roughly at sample time, heavy computation may impact performance.

3.6 Parallel runtime support

Requirements [R5](#) and [R6](#) call for OpenMP and MPI support. In original Mitos this is delivered by the `mitoshooks` component (Section [2.6](#)). In new Mitos no equivalent component exists; the functionality is provided entirely by Caliper’s built-in services and the configuration toggle in `Mitos_enable_mpi`.

OpenMP Support follows from Caliper’s OMPT integration combined with the per-thread initialization already present in the `libpfm` service. The `libpfm` service connects to the channel’s `create_thread_evt` and, for each new thread, opens its own `perf_event_open` descriptor, maps a ring buffer, and installs a SIGIO handler; the matching teardown runs on `release_thread_evt`. Caliper’s OMPT service translates OMPT thread-begin and thread-end callbacks into exactly these channel events, so every OMP worker spawned by the runtime becomes a sampling target through the `libpfm` service.

MPI Support is provided by Caliper’s `mpi` and `mpireport` services, which are enabled in the channel via `Mitos_enable_mpi`. The `mpi` service wraps MPI initialization and finalization through PMPI and tags every snapshot with the `mpi.rank` attribute. The `mpireport` service aggregates caliper records, first locally by rank and then globally across ranks, and emits a `cali` file containing the data from all ranks.

Caliper also ships integrations for Kokkos, CUDA, ROCm, and RAJA (Section [2.5](#)). These services do not drive sampling; they expose runtime-level events like kernel launches, region begin and end markers, host–device transfers as Caliper snapshots. Because they share the same channel and snapshot infrastructure as the `libpfm` service, a new Mitos channel can additionally enable any of them to correlate hardware samples with higher-level runtime events. This composition requires no changes to new Mitos.

Two issues surfaced in practice. First, the `libpfm` service carries thread-local state per open sampler, so threads that exit without going through `release_thread_evt` (i.e., threads not managed by the OMPT runtime) leak descriptors. Second, sampling must be started after `MPI_Init`, because the `mpi.rank` attribute is only populated once MPI is initialized; `Mitos_begin_sampler` therefore needs to be called from within the MPI region.

The implementation effort in new Mitos for parallel runtime support is the `Mitos_enable_mpi` toggle and nothing else. This is a concrete instance of the framework leverage argued in

Section 3.3.

3.7 Output format and MemAxes integration

The new implementation aims to replicate the output format of original Mitos, which is a CSV file containing one row per sample with columns for the sample's timestamp, the sampled event's name, the instruction pointer (IP) at which the event occurred, and any additional data attached by the user callback. `hwloc` hardware topology information is collected separately, just like original Mitos. Source files are copied based on the IP-to-source mapping provided by the `symbollookup` service.

3.8 Build and deployment

The new implementation is accompanied by a spack package. Therefore, the installation is very easy.

4 Evaluation

This chapter evaluates the new Mitos against the design requirements of Section 3.1. Requirements R3, R4 and R10 (interface) were evaluated qualitatively against the original Mitos C API in Section 3.1. Requirements R5–R9 are evaluated as functional checks (Section 4.2). Requirements R1, R2, R7 are evaluated as data-fidelity comparisons (Section 4.3); R16 and R17 as performance comparisons (Section 4.4). Requirements R11–R15 (microarchitecture independence) are argued by design and supported empirically by the cross-platform replication of the same evaluation (Section 4.5).

4.1 Methodology

Platforms. Two systems are used: an Intel Skylake-SP system (Xeon Bronze 3106) and an Intel Ice Lake-SP system (Xeon Platinum 8360Y), both exercising the Intel PEBS path. Both Mitos variants are expected to work on both systems. Ice Lake was originally identified as a platform on which original Mitos did not run, which was a big part of the initial motivation for the work, but a later investigation found that it does in fact run there; the cause of the earlier failure is unknown and was not reproduced in time for this evaluation. The Ice Lake column therefore reports a full comparison rather than a one-sided measurement, and R13 (PEBS on Ice Lake) is verified by demonstrating that new Mitos produces the expected output on Ice Lake. An AMD system was originally planned in order to exercise both IBS backends (R12); two attempts on different machines were made, but neither completed. One machine did not grant the necessary `kernel.perf_event_paranoid` permissions, and the Caliper installation on the other ran into Spack-related build issues that could not be resolved in time. The empirical IBS column is therefore absent from the result tables; the IBS claim rests on the design argument in Section 4.5 alone, and the consequences for R12 are listed in Section 4.6.

Workload. A naive triple-nested matrix multiplication, in serial, OpenMP-parallel, and MPI-parallel variants is profiled under each Mitos variant. The `matmul` kernel is shipped as an example in original Mitos’s source tree, and was simply adapted for the new implementation. The computation stays exactly the same. Compilation flags are `-O3 -g -std=c++17`; `-g` is required by R7. The matrix dimension is fixed at $N = 1024$ for the principal results; OpenMP uses 4 threads, MPI uses 4 ranks.

Procedure. For each cell of the experimental grid (platform $\times$ implementation variant $\times$ Mitos variant), $R = 20$ repetitions are run, preceded by a repetition as warm-up which is discarded. The machine is otherwise idle. For each cell the median across surviving repetitions is reported.

Configuration parity. For comparisons regarding R1 and R2, the two Mitos variants are configured to sample the same period. The kernel-level sample source is therefore held constant by construction, and any observed difference in the sample stream is attributable to the new code path (libpfm4, Caliper services, and `cali_to_csv.py`).

4.2 Functional checks (R5–R9)

Section 4.2 reports the pass/fail status of each functional requirement on each platform, for both Mitos variants.

R8 (sample callback) was not tested as part of this. This is because the implementation differs. The new Mitos creates a new Caliper Attribute for the collected value, which is not output by default, as to not deviate from the output format of the original Mitos. For this implementation, the callback functionality works as described previously.

R9 (MemAxes format) was not tested. The new implementation replicates the output format generated by the original Mitos, including a `hwloc`-provided `hardware.xml` and copying of source files to a subdirectory. Compatibility is therefore assumed but not verified.

Req	What is checked	Skylake		Ice Lake	
		orig	new	orig	new
R5	every OpenMP worker contributes samples	yes	no	yes	no
R6	every MPI rank contributes samples	yes	no	yes	no
R7	sample carries non-empty source/line	yes	yes (seq, omp)	yes	yes (seq, omp)
R9	MemAxes loads the output without errors	yes	unsure	yes	unsure

The serial path runs end-to-end for both Mitos variants on both platforms; the differences there are quantitative (sample counts, function-name overlap) rather than functional, and are taken up in Section 4.3. The OpenMP path collects samples on the new Mitos but only from the master thread; the MPI path performed similarly at one point in time but currently collects no samples at all.

4.3 Data (R1, R2, R7)

Table 4.1 reports, per cell, the median sample count across surviving repetitions for both Mitos variants, the cross-tool top-10 function Jaccard, and the source-resolution rate.

Platform	Variant	n_samples		resolved %	
		orig	new	orig	new
Skylake	seq	5262	3146	99.2	100.0
Skylake	omp	3156	841	99.7	100.0
Skylake	mpi	10613	0	13.8	—
Ice Lake	seq	5293	3301	99.9	100.0
Ice Lake	omp	3238	845	99.9	100.0
Ice Lake	mpi	17242	0	26.3	—

Table 4.1: Data-fidelity metrics per platform and parallelism variant; medians across 20 repetitions. “—” in the new-Mitos resolved-% column indicates that no samples were collected.

The sample-count comparison gives the cleanest read on data quantity (R2). In the serial cells the new Mitos collects 60–62 % of the original’s samples (3146 of 5262 on Skylake, 3301 of 5293 on Ice Lake) at the same configured period — a substantial undercount. In the OpenMP cells the gap is much larger ($\sim 27\%$), but most of this is assumed to be a cascade effect from only the master thread contributing samples, so this is effectively one-thread sampling compared against four-thread sampling. The MPI cells are an outright failure.

Caliper consistently reports more distinct `(file, line)` tuples (3–10 vs. 1–2), which is consistent with preserved-vs-collapsed inlines. Whether the underlying samples agree on the hot function is therefore not directly answered.

Source-resolution (R7) is at 100 % on every new-Mitos cell where samples were collected, which matches or slightly exceeds the original on serial and OpenMP. I am unsure whether the poor MPI baseline at 13.8 % on Skylake and 26.3 % on Ice Lake is due to an issue with my tests.

4.4 Performance (R16, R17)

Table 4.2 reports median wall-clock runtime, for both Mitos variants.

For the serial workload, new-Mitos runtime is within 1 % of the original on Skylake (7.78 vs. 7.71 s) and within 7 % on Ice Lake (4.98 vs. 4.67 s); R17 is met on serial. For OpenMP, new Mitos’s runtime is sharply lower than the original (2.83 vs. 7.82 on Skylake; 1.65 vs. 4.73 on Ice Lake), but the gap is likely due to improper parallelization for the

Platform	Variant	Runtime [s] (orig/new)
Skylake	seq	7.71 / 7.78
Skylake	omp	7.82 / 2.83
Skylake	mpi	4.09 / 3.92
Ice Lake	seq	4.67 / 4.98
Ice Lake	omp	4.73 / 1.65
Ice Lake	mpi	2.83 / 2.76

Table 4.2: Performance metrics per platform and parallelism variant; medians across 20 repetitions. The OpenMP runtime advantage for new Mitos make it seem like the original Mitos was not parallelized, but this is unverified.

original Mitos. This is not a real performance improvement. For MPI, runtimes are comparable (within $\sim 5\%$ on both platforms), but new Mitos’s runtime is over a workload that collects no samples, so the comparison says nothing about real sampling overhead.

4.5 Microarchitecture independence (R11–R15)

The microarchitecture-independence requirements are not amenable to direct measurement on a finite set of platforms: the universal claim “works on architectures I have not tested” cannot be empirically discharged. I therefore argue that the design meets the requirements (aside from the other reported issues), and substantiate the argument with the cross-platform replication reported in the preceding sections.

Argument by design. New Mitos contains no vendor-specific PMU encoding in its own source. The PMU event string is interpreted by libpfm4 (Section 2.4), which maintains compiled-in descriptor tables for Intel, AMD, ARM, and several other vendors and translates the same human-readable event string to the appropriate `perf_event_attr` encoding per microarchitecture. Adding support for a new microarchitecture in the same vendor family typically requires only a libpfm4 update; adding a new sampling backend (e.g. ARM SPE) requires a small configuration entry in the Caliper libpfm service. Original Mitos’s hard-coded `perf_event_attr` construction (Section 2.6) precludes this.

Empirical support. The same source compiles, links, and produces output of the documented shape on both test platforms (Section 4.2). The Skylake and Ice Lake runs share the PEBS code path and differ only in the libpfm4-emitted encoding. R13 (PEBS on Ice Lake) is verified by demonstrating that new Mitos produces the expected output on Ice Lake. R12 (Intel PEBS and AMD IBS support) is met for PEBS empirically and for IBS by design only, as the AMD system test was not completed (Section 4.6). Both

tested microarchitectures are Intel, so the empirical evidence speaks to within-vendor portability; the cross-vendor claim rests on the design argument alone.

4.6 Threats to validity

- AMD/IBS path not tested. R12 (Intel PEBS and AMD IBS support) is verified for PEBS only. Two attempts to test new Mitos on AMD systems failed. The IBS half of R12 therefore rests on the design argument in Section 4.5 alone.
- MPI and OpenMP tested separately. No MPI+OpenMP test was run, so the claim that new Mitos supports both in the same run is unverified. The MPI and OpenMP paths were tested separately, and both failed to reach feature parity with the original Mitos; the combined path is therefore not expected to work well either.
- Architecture independence is argued, not proved. Only two microarchitectures, both Intel, are tested. The claim about future hardware rests on the libpfm4 abstraction layer (Section 2.4); the empirical work substantiates it only insofar as the same source compiles and runs without modification on both test platforms.
- Cross-tool Top-10 Jaccard is unreliable. The cross-tool Jaccard is 0 in every cell, which is implausible given matching workloads; this very likely reflects a name-normalisation mismatch in the analyzer rather than disjoint sampling. R1 (data quality) therefore lacks a strong quantitative measure in this evaluation.

5 Conclusion

To conclude, this thesis did not achieve its original goal of providing a more robust, future-proof Mitos implementation. The new implementation is severely lacking compared to the original Mitos. It is briefly argued that the design is sound and the implementation is a first step towards a more robust, future-proof Mitos, but the current state of the codebase is not fit for production use.

List of Figures

2.1	Skid in interrupt-based versus precise sampling.	6
2.2	PEBS and IBS data flow.	7
3.1	Call stack on user-callback dispatch.	22

List of Tables

3.1	Comparison of the different technology candidates towards the design requirements.	18
3.2	new Mitos configuration API. The calls are ranked in terms of importance.	20
4.1	Data-fidelity metrics per platform and parallelism variant; medians across 20 repetitions. “—” in the new-Mitos resolved-% column indicates that no samples were collected.	28
4.2	Performance metrics per platform and parallelism variant; medians across 20 repetitions. The OpenMP runtime advantage for new Mitos make it seem like the original Mitos was not parallelized, but this is unverified. . .	29

Bibliography

- [1] C. Tebaldi, K. Debeire, V. Eyring, E. Fischer, J. Fyfe, P. Friedlingstein, R. Knutti, J. Lowe, B. O'Neill, B. Sanderson, D. van Vuuren, K. Riahi, M. Meinshausen, Z. Nicholls, K. B. Tokarska, G. Hurtt, E. Kriegler, J.-F. Lamarque, G. Meehl, R. Moss, S. E. Bauer, O. Boucher, V. Brovkin, Y.-H. Byun, M. Dix, S. Gualdi, H. Guo, J. G. John, S. Kharin, Y. Kim, T. Koshiro, L. Ma, D. Olivié, S. Panickal, F. Qiao, X. Rong, N. Rosenbloom, M. Schupfner, R. Séférian, A. Sellar, T. Semmler, X. Shi, Z. Song, C. Steger, R. Stouffer, N. Swart, K. Tachiiri, Q. Tang, H. Tatebe, A. Voldoire, E. Volodin, K. Wyser, X. Xin, S. Yang, Y. Yu, and T. Ziehn, "Climate model projections from the scenario model intercomparison project (scenariomip) of cmip6," *Earth System Dynamics*, vol. 12, no. 1, pp. 253–293, Mar. 2021. [Online]. Available: <https://esd.copernicus.org/articles/12/253/2021/> [Accessed: 2025-03-09]
- [2] J. Tatemura, T. Zou, J. Sankaranarayanan, Y. Huang, J. Chen, Y. Zhang, K. Lai, H. Zhang, G. N. B. Manoharan, G. Graefe, D. Agrawal, B. Adelberg, S. Kolhar, and I. Roy, "Progressive partitioning for parallelized query execution in google's napa," in *49th International Conference on Very Large Data Bases*, vol. 16, no. 12. Association for Computing Machinery (ACM), Aug. 2023, pp. 3475–3487. [Online]. Available: <https://doi.org/10.14778/3611540.3611541> [Accessed: 2025-03-09]
- [3] A. Siegel, E. W. Draeger, J. Deslippe, T. Evans, M. M. Francois, T. C. Germann, D. F. Martin, and W. Hart, "Map applications to target exascale architecture with machine-specific performance analysis, including challenges and projections," Exascale Computing Project, Tech. Rep. ORNL/TM-2021/2103, Mar. 2021. [Online]. Available: <https://www.osti.gov/biblio/1838979> [Accessed: 2025-03-09]
- [4] P. D. Vecchio. (2022, Aug.) Demystifying software performance optimization. Intel. [Online]. Available: <https://www.intel.com/content/www/us/en/developer/articles/technical/demystifying-software-performance-optimization.html> [Accessed: 2026-05-10]
- [5] Intel Corporation, *Intel® 64 and IA-32 Architectures Software Developer's Manual Combined Volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, and 4*, 91st ed., Mar. 2026. [Online]. Available: <https://www.intel.com/content/www/us/en/content-details/916575/>

- [intel-64-and-ia-32-architectures-software-developer-s-manual-combined-volumes-1-2a-2b-2c-2d-3a-3b-3c.html](#) [Accessed: 2026-04-27]
- [6] Advanced Micro Devices, Inc., *AMD64 Architecture Programmer's Manual, Volumes 1-5*, 4th ed., Mar. 2026. [Online]. Available: https://docs.amd.com/v/u/en-US/40332_4.09_APM_PUB [Accessed: 2026-04-27]
- [7] The Mitos Contributors, "Mitos," GitHub. [Online]. Available: <https://github.com/caps-tum/mitos/tree/master> [Accessed: 2026-04-29]
- [8] A. Giménez, T. Gamblin, B. Rountree, A. Bhatele, I. Jusufi, P.-T. Bremer, and B. Hamann, "Dissecting on-node memory access performance: A semantic approach," in *SC '14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2014, pp. 166–176.
- [9] J. L. Hennessy and D. A. Patterson, *Computer Architecture, Fifth Edition: A Quantitative Approach*, 5th ed. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2011.
- [10] B. Cohen, M. Subramony, and M. Clark, "Next generation"zen 5" core," in *Hot Chips 2024*, 2024. [Online]. Available: https://hc2024.hotchips.org/assets/program/conference/day2/24_HC2024.AMD.Cohen.Subramony.final.pdf [Accessed: 2026-04-21]
- [11] Wlzzard, "Intel panther lake technical deep dive," *techpowerup.com*, Oct. 2025. [Online]. Available: <https://www.techpowerup.com/review/intel-panther-lake-technical-deep-dive/> [Accessed: 2026-04-21]
- [12] M. Bruce, "Arm neoverse v2 platform: Leadership performance and power efficiency for next-generation cloud computing, ml and hpc workloads," in *Hot Chips 2023*. ARM, 2023. [Online]. Available: <https://hc2023.hotchips.org/assets/program/conference/day1/CPU1/HC2023.Arm.MagnusBruce.v04.FINAL.pdf> [Accessed: 2026-04-21]
- [13] C. Lam, "Analyzing zen 2's cinebench r15 lead," *Chips and Cheese*, Feb. 2021, accessed 25.3.2026. [Online]. Available: <https://chipsandcheese.com/p/analyzing-zen-2s-cinebench-r15-lead> [Accessed: 2026-03-25]
- [14] S. Mittal, "A survey of techniques for dynamic branch prediction," *Concurrency and Computation: Practice and Experience*, vol. 31, no. 1, 2018.
- [15] Advanced Micro Devices, *Software Optimization Guide for AMD Family 17h Processors*, 3rd ed., Feb. 2021. [Online]. Available: https://docs.amd.com/v/u/en-US/55723_3_01_0 [Accessed: 2026-05-08]

- [16] S. L. Harris and D. Harris, “8 - memory systems,” in *Digital Design and Computer Architecture*, S. L. Harris and D. Harris, Eds. Morgan Kaufmann, 2022, pp. 498–541. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/B9780128200643000088> [Accessed: 2026-04-20]
- [17] P. J. Denning, “The locality principle,” *Communications of the ACM*, vol. 48, no. 7, pp. 19–24, 2005.
- [18] J. Weinberg, M. McCracken, E. Strohmaier, and A. Snively, “Quantifying locality in the memory access patterns of hpc applications,” in *ACM/IEEE SC 2005 Conference (SC’05)*. IEEE, 2005, pp. 50–50.
- [19] A. Zandieh, M. Daliri, M. Hadian, and V. Mirrokni, “Turboquant: Online vector quantization with near-optimal distortion rate,” 2025.
- [20] M. H. Fisher, “Analysis of memory access patterns for large language model inference,” mathesis, Virginia Tech, Jul. 2025. [Online]. Available: <https://hdl.handle.net/10919/135946> [Accessed: 2026-04-29]
- [21] Arm Limited, *Arm® Architecture Reference Manual for A-profile architecture*, m.b ed., Apr. 2026. [Online]. Available: <https://developer.arm.com/documentation/ddi0487/mb> [Accessed: 2026-04-27]
- [22] Linux man-pages, *perf_event_open(2) System Calls Manual*, Jun. 2024.
- [23] Intel Corporation, *Intel® VTune™ Profiler User Guide*, 2025th ed., Jun. 2025. [Online]. Available: <https://www.intel.com/content/www/us/en/docs/vtune-profiler/user-guide/2025-4> [Accessed: 2026-04-27]
- [24] Advanced Micro Devices, Inc., *AMD uProf Getting Started Guide (Linux)*, 5th ed., Feb. 2026. [Online]. Available: <https://docs.amd.com/r/en-US/68658-uProf-getting-started-guide> [Accessed: 2026-04-27]
- [25] J. Mundichipparakkal, R. Chang, Y. Cai, D. Greene, M. Williams, and T. Chouhan, “Arm statistical profiling extension: Performance analysis methodology,” Arm, techreport 109429, Oct. 2023. [Online]. Available: <https://developer.arm.com/documentation/109429/0100/> [Accessed: 2026-04-28]
- [26] M. A. Sasongko, M. Chabbi, P. H. J. Kelly, and D. Unat, “Precise event sampling on amd versus intel: Quantitative and qualitative comparison,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 5, pp. 1594–1608, May 2023.

- [27] Intel, *Intel® Microarchitecture Codename Nehalem Performance-Monitoring Unit Programming Guide*, 2010. [Online]. Available: <https://www.intel.com/content/dam/develop/external/us/en/documents/30320-nehalem-pmu-programming-guide-core.pdf> [Accessed: 2026-04-28]
- [28] P. J. Drongowski, “Instruction-based sampling: A new performance analysis technique for amd family 10h processors,” Nov. 2007. [Online]. Available: https://www.amd.com/content/dam/amd/en/documents/archived-tech-docs/white-papers/AMD_IBS_paper_EN.pdf [Accessed: 2026-04-29]
- [29] J. Greathouse, S. Carnà, and W. Cong, “Amd research instruction based sampling toolkit,” GitHub, 2021. [Online]. Available: https://github.com/jlgreathouse/AMD_IBS_Toolkit/tree/4d4884b9bf1f20fc096270a7d071bac03564a636 [Accessed: 2026-04-29]
- [30] J. Dean, J. Hicks, C. Waldspurger, W. Wehl, and G. Chrysos, “Profileme: hardware support for instruction-level profiling on out-of-order processors,” in *Proceedings of 30th Annual International Symposium on Microarchitecture*, 1997, pp. 292–302.
- [31] S. Eranian *et al.*, “libpfm4,” SourceForge. [Online]. Available: <https://sourceforge.net/p/perfmon2/libpfm4/> [Accessed: 2026-04-30]
- [32] S. Eranian, “Perfmon2: a flexible performance monitoring interface for linux,” in *Proc. of the 2006 Ottawa Linux Symposium*, 2006, pp. 269–288.
- [33] H. Jagode, A. Danalis, G. Congiu, D. Barry, A. Castaldo, and J. Dongarra, “Advancements of papi for the exascale generation,” *The International Journal of High Performance Computing Applications*, vol. 39, no. 2, pp. 251–268, Dec. 2024.
- [34] D. Boehme, T. Gamblin, D. Beckingsale, P.-T. Bremer, A. Gimenez, M. LeGendre, O. Pearce, and M. Schulz, “Caliper: Performance introspection for hpc software stacks,” in *SC ’16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2016, pp. 550–560. [Online]. Available: <https://ieeexplore.ieee.org/document/7877125> [Accessed: 2026-03-23]
- [35] The Caliper contributors. Caliper. GitHub. [Online]. Available: <https://github.com/llnl/Caliper> [Accessed: 2026-03-23]
- [36] —, *Caliper: A Performance Analysis Toolbox in a Library*, 2.15.0 ed., Lawrence Livermore National Laboratory. [Online]. Available: <https://software.llnl.gov/Caliper/> [Accessed: 2026-05-11]

- [37] Message Passing Interface Forum, “MPI: A Message-Passing Interface Standard, Version 4.1,” Message Passing Interface Forum, Tech. Rep., Nov. 2023, chapter 15: The MPI Tool Information Interface; Section on the Profiling Interface (PMPI). [Online]. Available: <https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf> [Accessed: 2026-05-04]
- [38] OpenMP Architecture Review Board, *OpenMP Application Programming Interface*, 6th ed., OpenMP, Nov. 2024. [Online]. Available: <https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-6-0.pdf> [Accessed: 2026-05-08]
- [39] C. R. Trott, D. Lebrun-Grandié, D. Arndt, J. Ciesko, V. Dang, N. Ellingwood, R. Gayatri, E. Harvey, D. S. Hollman, D. Ibanez, N. Liber, J. Madsen, J. Miles, D. Poliakoff, A. Powell, S. Rajamanickam, M. Simberg, D. Sunderland, B. Turcksin, and J. Wilke, “Kokkos 3: Programming model extensions for the exascale era,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 4, pp. 805–817, 2022.
- [40] NVIDIA Corporation, *CUDA Programming Guide*, 13rd ed., Mar. 2026. [Online]. Available: https://docs.nvidia.com/cuda/archive/13.2.0/pdf/CUDA_C_Programming_Guide.pdf
- [41] Advanced Micro Devices, Inc., *AMD ROCm™ Programming Guide*, 7th ed., May 2026. [Online]. Available: https://rocm-handbook.amd.com/_/downloads/amd-rocm-programming-guide/en/docs-7.2.3/pdf/ [Accessed: 2026-05-08]
- [42] NVIDIA Corporation, *The user manual for NVIDIA profiling tools for optimizing performance of CUDA applications.*, 12nd ed., May 2025. [Online]. Available: https://docs.nvidia.com/cuda/pdf/CUDA_Profiler_Users_Guide.pdf [Accessed: 2026-05-08]
- [43] —, *Nsight Compute Documentation*, 2026th ed., 2026. [Online]. Available: <https://docs.nvidia.com/nsight-compute/> [Accessed: 2026-05-01]
- [44] M. Geitner, “Analysis and implementation of amd ibs sample collection for performance analysis purposes,” Master’s thesis, Technical University of Munich (TUM), Jul. 2023.
- [45] S. Vanecek, M. Turner, M. Gajbe, M. Wolf, and M. Schulz, “Modeling the potential of message-free communication via cxl.mem,” in *Proceedings of the Supercomputing Asia and International Conference on High Performance Computing in Asia Pacific Region*, ser. SCA/HPCAsia 2026. ACM, Jan. 2026, pp. 258–270.
- [46] OpenMP Architecture Review Board, “OpenMP Application Programming Interface, Version 5.2,” OpenMP Architecture Review Board, Tech. Rep., Nov. 2021, chapter

- 4: OMPT Interface. [Online]. Available: <https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-5-2.pdf> [Accessed: 2026-05-04]
- [47] F. Broquedis, J. Clet-Ortega, S. Moreaud, N. Furmento, B. Goglin, G. Mercier, S. Thibault, and R. Namyst, “hwloc: A Generic Framework for Managing Hardware Affinities in HPC Applications,” in *2010 18th Euromicro Conference on Parallel, Distributed and Network-based Processing*, 2010, pp. 180–186.
- [48] B. Buck and J. K. Hollingsworth, “An API for Runtime Code Patching,” in *The International Journal of High Performance Computing Applications*, vol. 14, no. 4, 2000, pp. 317–329.
- [49] S. Vanecek and M. Schulz, “sys-sage: A unified representation of dynamic topologies and attributes on hpc systems,” in *Proceedings of the 38th ACM International Conference on Supercomputing*, ser. ICS ’24. ACM, 2024, pp. 363–375.
- [50] DEEP-SEA, “Memaxes.” [Online]. Available: <https://deep-projects.eu/modular-supercomputing/software/benchmarking-and-tools/memaxes/> [Accessed: 2026-05-08]
- [51] D. Schenk, “Extension of memaxes samplevisualization to process amd ibs samples,” bathesis, Technical University of Munich, Aug. 2023. [Online]. Available: <https://mediatum.ub.tum.de/doc/1729631/1729631.pdf> [Accessed: 2026-05-08]
- [52] cppreference.com, “inline specifier.” [Online]. Available: <https://en.cppreference.com/cpp/language/inline> [Accessed: 2026-05-11]
- [53] Linux man-pages, *signal-safety(7)* — *Linux manual page*, Oct. 2025.